

Лабораторная работа №2

Постановка задачи

Написать программу на C++ для сравнения различных алгоритмов поиска. Написать краткое техническое задание (ТЗ). Выполнить реализацию. Написать для нее тесты.

Минимальные требования к программе. В программе должно быть реализовано несколько различных методов поиска. Для каждого метода существуют различные модификации, дополняющие или улучшающие возможности или характеристики метода. Они перечислены в таблице ниже. Каждая модификация снабжена рейтинговой оценкой, пропорциональной сложности реализации. Студент самостоятельно выбирает конфигурацию для реализации исходя из условия: суммарный рейтинг должен быть не меньше 55.

В качестве типа данных для поиска следует использовать информационные объекты с нетривиальной структурой. Например, класс, описывающий студента (Фамилия, Имя, Отчество, Номер группы, Дата рождения и др.). Поиск следует осуществлять по какому-либо атрибуту. Программа должна предоставлять возможность выбора атрибута. В узлах дерева, таким образом, следует хранить как значение ключа, так и указатель на соответствующий объект данных. В роли указателя может выступать индекс в последовательности.

Основные реализованные алгоритмы необходимо покрыть тестами. Программа должна позволять выбрать любой из реализованных алгоритмов поиска и запустить его на (достаточно произвольных) исходных данных. При этом должна быть возможность как автоматической, так и ручной проверки корректности работы алгоритмов (в т.ч. должна быть возможность просмотра как исходных данных, так и результата). Программа должна обладать пользовательским интерфейсом (консольным или графическим). Пользовательский интерфейс, в особенности, графический, тестировать не требуется. Программа должна предоставлять функцию измерения времени выполнения алгоритма. Должна быть функция сравнения алгоритмов – по времени выполнения на одних и тех же входных данных¹. Программа должна предоставлять функционал по построению графиков зависимостей, либо по выгрузке необходимых данных в открытых форматах (например, csv).

Методы поиска и их модификации

№	Модификация	Рейтинг
М-1.	Бинарный поиск по отсортированной последовательности (базовая реализация)	10
М-1.1.	Возможность выбора пропорции деления: пополам, либо в соответствии с «золотым сечением»	5
М-1.2.	Пропорции деления задаются параметром – парой натуральных чисел, либо номером последовательности Фибоначчи	5
М-2.	Бинарное дерево	15
М-2.1.	Сбалансированное	7
М-2.2.	Прошитое	7
М-2.3.	Поддержка интерфейса SortedSequence	10

¹ Следует рассматривать три основных случая: последовательность уже отсортирована в нужном направлении; последовательность отсортирована в обратном направлении; последовательность не отсортирована. В случае деревьев, имеется в виду сравнение времени построения дерева, а не время поиска в уже построенном дереве.

М-2.4.	Поддержка интерфейса <code>IDictionary</code>	10
М-2.5.	Хранение данных в структурированном бинарном файле	10
М-3.	В-дерево (без удаления)	20
М-3.1.	Операция удаления	15
М-3.2.	Поддержка интерфейса <code>SortedSequence</code>	20
М-3.3.	Поддержка интерфейса <code>IDictionary</code>	10
М-3.4.	Хранение данных в структурированном бинарном файле	20
М-4.	Хеш-таблица, реализация интерфейса <code>IDictionary<TKey, TElement></code> (без разрешения коллизий)	12
М-4.1.	Разрешение коллизий с помощью списков	7
М-4.2.	Разрешение коллизий с помощью смещения адресов	7

Пояснения.

- Деревья поиска следует реализовывать как шаблонные классы. Это позволит как хранить в них сами данные, так и лишь индексы, отмечающие положение данных в последовательности. Т.е. у шаблона два параметра-типа: тип ключа и тип хранимого значения.
 - В случае, если хранятся просто числа (скажем, вещественные), значения этих параметров совпадают: `double`, ключ и значение также совпадают.
 - Если дерево строится «над» числовой последовательностью и требуется хранить положение элемента в исходной последовательности, то тип хранимого значения будет `int`. Ключом будет число, а значением – его индекс в последовательности.
 - Если последовательность состоит из сложных информационных объектов (например, класс `Student`), а поиск ведется по атрибуту типа `T`, то дерево будет инстанцировано с параметрами шаблона: `Tree<T,int>` (в предположении, что хранимым является именно индекс объекта в последовательности), либо `Tree<T,Student*>` (если в дереве хранятся именно указатели).
- Алгоритм построения дерева из последовательности может быть реализован в одном из следующих вариантов:
 - Как статический метод класса, описывающего дерево (не желательно, т.к. создает жесткую связку между классом дерева и классом последовательности).
 - В составе отдельного класса (например, `BinaryTreeBuilder`). Это позволит в дальнейшем легко расширять функциональность этого класса, добавляя методы для построения деревьев из других структур данных (например, «голых» массивов или `std::vector<>`).
 - В виде отдельной функции.
- В случае с хранением данных в бинарном файле, дерево хранит лишь индексы – положение данных в файле.
- Краткая спецификация `SortedSequence`:

Класс <code>SortedSequence<TElement></code>		
Является модификацией интерфейса <code>Sequence</code> , описывающей отсортированные последовательности. Основное отличие состоит в том, что методы <code>Append</code> , <code>Prepend</code> и <code>InsertAt</code> заменены одним методом <code>Add</code> .		
Название	Сигнатура	Описание
Атрибуты		
<code>Length</code>	<code>int GetLength()</code>	Длина последовательности (количество элементов)

IsEmpty	<code>int GetIsEmpty()</code>	Признак того, является ли последовательность пустой
Методы		
Get	<code>TElement Get(int index)</code>	Получение элемента по индексу.
GetFirst	<code>TElement GetFirst()</code>	Получить первый элемент последовательности
GetLast	<code>TElement GetLast</code>	Получить последний элемент последовательности
IndexOf	<code>int IndexOf(TElement element)</code>	Получить индекс элемента последовательности. Если указанного элемента в последовательности не содержится, возвращается значение -1.
GetSubsequence	<code>SortedSequence<TElement> GetSubsequence(int startIndex, int endIndex)</code>	Получить подпоследовательность: начиная с элемента с индексом startIndex и заканчивая элементом с индексом endIndex
Add	<code>Void Add(TElement element)</code>	Добавить элемент в последовательность. Элемент автоматически вставляется так, что итоговая последовательность остается отсортированной.

5. Хеш-таблица должна хранить данные в структуре данных, обеспечивающих обращение по индексу (index-based addressing) – например, Sequence. Хеш-функция должна вычислять положение в этом массиве на основании ключа. Это достигается в 2 этапа: сначала для каждого используемого типа ключа T выбирается некоторая функция $\text{GetHashCode}: T \rightarrow \text{int}$, которая для каждого экземпляра типа T возвращает некоторое положительное целое (int). Эта функция должна удовлетворять следующему обязательному условию:

Если значения этой функции для некоторых двух элементов различаются, то и сами элементы различны.

Полученное таким образом целое число является промежуточным для вычисления итогового значения хеш-функции. Хеш-функция также должна удовлетворять приведенному выше обязательному условию. Значение хеш-функции трактуется как индекс элемента в внутреннем массиве хеш-таблицы.

В минимальном варианте необходимо реализовать хотя бы одну из следующих возможностей:

- использовать какую-либо списковую структуру (например, реализованный ранее тип данных Sequence) для обработки коллизий;
- автоматическую перестройку хеш-функции и внутренней последовательности при возникновении коллизий.

Для получения максимального балла необходимо реализовать обе указанные возможности, а также автоматическую перестройку при удалении элементов.

6. Перестройку хеш-таблицы необходимо проводить не при каждом удалении, а лишь тогда, когда число элементов стало ниже некоторого порогового значения. Более точно, следует зафиксировать два (необязательно целых) положительных числа: p и q (причем $p \geq q > 1$). При этом хеш-таблица характеризуется двумя неотрицательными целыми числами: количеством элементов (n) и вместимостью (c). Всякий раз, когда оказывается, что $n \leq c/p$, следует сжать в q раз. Если же оказывается, что $n = c$, то таблицу следует расширить в q раз.
7. Краткая спецификация IDictionary:

Класс IDictionary<TKey, TElement>		
Название	Сигнатура	Описание
Атрибуты		
Count	int GetCount()	Количество элементов, которое фактически хранится
Capacity	int GetCapacity()	Вместимость – максимальное количество элементов, которое можно поместить в таблицу без необходимости ее перестройки.
Методы		
Get	TElement Get(TKey key)	Получение элемента по ключу. Выбрасывает исключение, если элемента не существует.
ContainsKey	BOOL GetFirst(TKey key)	Проверка, что в таблице уже есть элемент с заданным ключом.
Add	void Add(TKey key, TElement element)	Добавить элемент с заданным ключом.
Remove	void Remove(TKey key)	Удаляет элемент с заданным ключом. Выбрасывает исключение, если заданный ключ отсутствует в таблице.

Пример. Будем искать студентов по ФИО (FullName).

```
int GetHashCode(string s) { /*...*/ }
Dictionary<string, Student> students(&GetHashCode, 25);
students.Add(student1.GetFullName(), student1);
students.Add(student2.GetFullName(), student2);
students.Add(student2.GetFullName(), student3);
// students.ContainsKey(student1.GetFullName()) => TRUE
// students.ContainsKey(student4.GetFullName()) => FALSE
students.Get(student1.GetFullName()) // => student1
students.ContainsKey(student4.GetFullName()) // => Exception
students.GetCount() // => 3
```

students.GetCapacity() // => 25, если автоматической перестройки
 // нет и 25/q, если есть

Критерии оценки

1.	Качество программного кода:	– стиль (в т.ч.: имена, отступы и проч.) (0-2) – структурированность (напр. декомпозиция сложных функций на более простые) (0-2) – качество основных и второстепенных алгоритмов (напр. обработка граничных случаев и некорректных исходных данных и т.п.) (0-3)	0-7 баллов
2.	Качество пользовательского интерфейса:	– предоставляемые им возможности (0-2) – наличие ручного/автоматического ввода исходных данных (0-2) – настройка параметров для автоматического режима отображение исходных данных и промежуточных и конечных результатов и др. (0-2)	0-6 баллов
3.	Качество тестов	– степень покрытия – читаемость – качество проверки (граничные и некорректные значения, и др.)	0-3 баллов
4.	Полнота выполнения задания и качество ТЗ	Оценивается качество подготовки ТЗ, полнота выполнений минимальных требований	0-5 баллов
5.	Владение теорией	знание алгоритмов, области их применимости, умение сравнивать с аналогами, оценить сложность, корректность реализации	0-3 баллов
6.	Оригинальность реализации	оцениваются отличительные особенности конкретной реализации – например, общность структур данных, наличие продвинутых графических средств, средств ввода-вывода, интеграции с внешними системами и др.	0-9 баллов
Итого			0-33 баллов

Для получения зачета за выполнения лабораторной работы необходимо соблюдение всех перечисленных условий:

- оценка за п. 1 должна быть не менее 3 баллов
- оценка за п. 4 должна быть не менее 3 баллов
- оценка за п. 5 должна быть больше 0
- суммарная оценка за работу без учета п. 6 должна быть не менее 15 баллов